7.4.3 File upload forms and views

You've added an `ImageField` to the `Venue` model and, through the Django Admin, have uploaded a picture. The Django Admin is for, well, admins, so to create a self-service version of this, you need a new view. Since the data getting filled in is for a model, it makes sense to use a `ModelForm`. The `Venue` object is in the `bands` app, which means you'll need a new file: RiffMates/bands/forms.py. Inside this file, add the code in the following listing, which uses the `modelform_factory` shortcut to quickly build a form based on the `Venue` model.

Listing 7.17 Create a form for a `Venue` using `modelform_factory`

```
# RiffMates/bands/forms.py
from django import forms
from bands.models import Venue

VenueForm = forms.modelform_factory(Venue,
    fields=["name", "description", "picture"])
```

The factory function returns a ModelForm class and takes the associated model class as well as a list of fields as arguments.

The VenueForm contains name, description, and picture fields.

Aren't `ModelForms` wonderful? Two lines of code (one if you aren't old school like me and insist files are restricted to 80 columns), and you're ready to take data from the user and populate your model.

The view for this form is very similar to the one you wrote to input a classified ad, but in this case the security check is a little different. Recall that users get associated with venues through the `venues_controlled` many-to-many relationship inside the `UserProfile` object. To keep things simple, anyone who is logged in can add a `Venue`, but only those with a corresponding `venues_controlled` entry can edit one.

Like with the classified view, this view does double duty: a venue ID of `0` results in adding a new venue, while a larger venue ID performs an edit. The code after the security check is almost identical. The HTTP mechanism for uploading files is slightly different from regular form fields, and the associated file isn't in the `request.POST` attribute but in `request.FILES`. The form constructor knows how to handle this, and you simply pass it in with the other arguments.

This time around, you don't need to modify the contents of `VenueForm` before creating the associated `Venue` object, so the form's `save()` method no longer needs the `commit=False` argument. One last step is to add the newly created `Venue` object to the user's `venues_controlled` relationship. The `add()` method on a `ManyToManyField` is smart enough to disallow adding something twice, so you don't even have to check if the relationship already exists.

Add the code in the following listing to RiffMates/bands/views.py, and then afterward, register a route for both adding and editing a venue. You can register the route twice with two different names: the first for adding a venue using the default value for `venue_id` (and thus not requiring a URL argument) and the second for editing the